大模型训练框架与基础设施 前置知识

> 这份文档为零基础读者准备。如果你已经熟悉分布式训练的基本概念，可以直接翻阅主报告。

一、为什么大模型需要特殊的训练框架?

训练一个大模型和训练一个小模型完全不同。一个最简单的比喻：

训练小模型像是做一顿饭——一个人、一个灶台、一口锅就搞定了。

训练大模型像是给一个万人运动会做饭——你需要多个厨房、多个厨师、分工协作。有人负责切菜、有人掌勺、有人传菜。水电气要通到每个灶台，食材要从仓库运到各个厨房。

大模型训练框架就是解决这个"多人协作"问题的工具。

二、并行训练：什么是"并行"?

"并行"就是把一个任务拆成多个部分同时做。大模型训练中有几种不同的并行方式：

Data Parallelism（数据并行）

最直观的并行方式。想象你在参加一个考试，但你的朋友也同时参加——你们各做一份试卷，做完后对答案，把答案平均一下作为最终答案。

训练中就是：N 张 GPU 各拿一批不同数据算梯度 → 所有 GPU 把梯度汇总平均 → 每张卡用平均后的梯度更新自己的模型。结果是所有卡片得到了相同的模型更新。

优点：简单，容易理解。

缺点：每张卡都要存一份完整的模型，模型太大时放不下。

Model Parallelism（模型并行）

当模型太大、单张 GPU 放不下时，把模型切开放在多张卡上。

想象一套房子装修的图纸太大，一张桌子铺不开，你把它分成客厅部分、卧室部分、厨房部分三张图分别放在三张桌子上。

训练中就是把神经网络的不同层放在不同的 GPU 上。数据先经过第一层所在的 GPU，算完传给第二层的 GPU，以此类推。

Tensor Parallelism（张量并行）

模型并行的"精细版"。不只是在"层"的层面切分，而是在**每一层内部**把权重矩阵切开。

一个 Transformer 层有个巨大的矩阵，把矩阵劈成两半，两张卡各算一半。相当于一个厨房里两个厨师——一个切菜的左半边，一个切菜的右半边。

Pipeline Parallelism（流水线并行）

把模型的不同层放在不同的卡上，数据像流水线一样流过。类似工厂流水线——每个人只做自己的那一道工序。

Sequence Parallelism（序列并行）

大模型处理长文本时，在"文本长度"这个维度上切分。A 卡处理文章的前半段，B 卡处理后半段。

Expert Parallelism（专家并行）

MoE（混合专家）模型的专属并行策略。MoE 模型中有多个"专家"子网络，每个 token 只激活少数专家。专家并行把这些专家分布到不同 GPU 上——每个 token 在计算时可能需要跨卡访问专家，这称为 **All-to-All 通信**。EP 在 2024-2026 年的 MoE 大模型（如 DeepSeek-V3、Mixtral）中被广泛使用。

NCCL——GPU 通信的"通用语"

NCCL（NVIDIA Collective Communications Library）是 NVIDIA 的高性能 GPU 通信库，所有主流框架（PyTorch、DeepSpeed、Megatron）底层都调它。AllReduce、All-to-All 等通信操作都通过 NCCL 完成。它的存在让用户不用手写 GPU 间的数据传输代码——框架替你调好了。

三、显存去哪了？

训练大模型时，GPU 的显存被三件事占用了：

1. **模型参数**：权重本身（比如 175B 参数的模型，仅参数就占 350GB）
2. **优化器状态**：Adam 优化器需要额外保存每个参数的"动量"和"方差"（又占 700GB）
3. **中间激活值**：前向计算过程中每一层的输出（显存黑洞，序列越长越大）

ZeRO（Zero Redundancy Optimizer）的核心理念：**既然这些数据在每张卡上都是重复的，不如切分它们，每张卡只存一份**。

四、ZeRO 的三个阶段（重要）

阶段	切分的内容	显存节省	通信量
Stage 1	优化器状态（动量、方差）	4×	基本不变
Stage 2	优化器状态 + 梯度	8×	略有增加
Stage 3	优化器状态 + 梯度 + 参数	N×（GPU 数）	增加约 50%

一个具体的例子

假设训练一个 175B 参数的模型，用 Adam 优化器，fp16 精度：

- **不用 ZeRO**：每张卡需要约 3,850 GB 显存（远超任何 GPU 的容量）
- **ZeRO Stage 3 配 64 张卡**：每张卡只需要约 60 GB（A100-80G 装得下）

这就是 ZeRO 的魔力——它让超大模型训练变得在普通 GPU 集群上可行。

ZeRO-Offload

如果显存还是不够，把一部分数据"卸"到 CPU 内存。GPU 只做计算密集的前向和反向传播，优化器更新在 CPU 上完成。

ZeRO-Infinity

更进一步，可以在 GPU 显存、CPU 内存、NVMe SSD 三级存储之间调度数据。打个比方：GPU 显存是你的桌面，CPU 内存是你的书架，NVMe 是你的地下室仓库——不常用但又不能丢的东西放地下室，用到的时候再拿出来。

五、DDP vs FSDP vs DeepSpeed——有什么区别?

技术	做什么	谁做的	关系
DDP	数据并行的基本实现	PyTorch	每张卡存完整模型，梯度同步平均
FSDP	数据并行 + 参数分片	PyTorch（Meta）	等价于 ZeRO Stage 3，原生集成 PyTorch
DeepSpeed	全套优化方案	微软	ZeRO + Offload + MoE + RLHF 一站式

FSDP 是 PyTorch 对 ZeRO Stage 3 的重新实现。两者功能类似，但 FSDP 跟 PyTorch 生态捆绑更紧（更好兼容 torch.compile 等），DeepSpeed 则有更多高级功能（Offload、MoE、通信优化）。

六、FlashAttention——让注意力计算变快的关键技术

Attention 是 Transformer 的核心操作。计算 Attention 需要访问一个巨大的 N×N 矩阵（N 是序列长度），这个矩阵大到没法全部放在 GPU 的高速缓存里，必须频繁读写慢速显存——就像频繁从冰箱取菜而不是用手边的备菜台。

FlashAttention 的洞察：把 Attention 计算切成小块，在 GPU 的高速缓存里一块一块算完，再写回显存。这样显存读写次数从 $O(N^2)$ 降到 $O(N)$。

三个版本的演进：

- **FlashAttention 1**（2022）：分块计算 + 不存完整 Attention 矩阵 → BERT 训练提速 15%
- **FlashAttention 2**（2023）：更好的并行策略 → 再提速 2×
- **FlashAttention 3**（2024）：利用新硬件特性（FP8 Tensor Core） → 再提速 1.5–2×

七、3D 并行——最常用的组合拳

实际训练大模型时，很少只用一种并行策略。最常用的是"3D 并行"：

并行策略	通信量	通常部署在	原因
Tensor Parallelism	通信密集（每层都有）	单台机器内部	NVLink 带宽极高
Pipeline Parallelism	通信量小（每层边界一次）	不同机器之间	网络带宽有限也能承受
Data Parallelism (ZeRO)	通信中等	机器之间	利用 ZeRO 减少内存冗余

示例：训练 175B 模型，用 64 张 GPU 的集群：

- 4 张卡一组做 Tensor Parallelism（在单台 DGX 内）
- 2 组串联做 Pipeline Parallelism
- 8 组并联做 Data Parallelism（ZeRO）

八、一张图总结

```
原始问题：175B 参数模型要 ~3850GB 显存，而 A100-80G 只有 80GB
                    ↓
              怎么解决？
                    ↓
┌──────────────────────────────────────────────────┐
│   三种并行 + ZeRO = 模型能训练                     │
│                                                    │
│  TP：把每层权重切半 → 两张卡一起算一层              │
│  PP：把各层分开放 → 流水线式算完整个网络            │
│  DP（ZeRO）：参数分片 → 每卡只存自己的那份          │
│  + FlashAttention → 长序列也能算                   │
└──────────────────────────────────────────────────┘
                    ↓
          最终在 64 张 A100 上训练成功
```

另一个直觉：**训练框架 = 多卡协作的"操作系统"**。它管"调度"（哪张卡算什么）、"通信"（卡之间怎么传数据）、"存储"（参数放哪里、激活值放哪里），让用户不用操心这些底层细节，只需要写模型代码。

通信原语：多卡之间怎么"对话"

多张 GPU 一起训练模型时，需要频繁交换数据。几种核心的"通信原语"：

- **AllReduce**：每张卡有自己计算出的梯度，需要把所有卡的梯度加在一起取平均，然后同步回每张卡。相当于"所有人报数，算完总和后告诉每个人"
- **AllGather**：每张卡有数据的一部分，把所有部分拼接起来，发回每张卡
- **ReduceScatter**：先求和，再分片发回——Combine 操作加上 Scatter 操作

Pipeline Bubble（流水线气泡）

流水线并行的一个固有效率损失。当第一张卡在处理第一个 batch、最后一张卡在等待时，中间有些卡处于空闲状态——这个等待的空隙就叫 bubble。bubble 越小，流水线效率越高。解决方案通常是增加 **micro-batch** 数量（把一个大 batch 切成很多小份，让流水线填得更满）。

Optimizer States（优化器状态）

训练模型时，优化器（如 Adam）不仅要存储模型参数，还要额外存储两个状态：梯度的一阶矩（动量）和二阶矩（方差的累积）。一个 70B 参数的模型，用 Adam 优化器需要额外占用约 16 字节/参数 × 70B ≈ 1.1TB 显存——比模型本身还大。这就是为什么 ZeRO 优化器要把这些状态也分片存储在多张卡上。